

4.Multimodal visual understand speech interaction

4.Multimodal visual understand speech interaction

1. Concept Introduction
 - 1.1 What is "Visual Understanding"?
 - 1.2 Implementation Principle Overview
2. Code Explanation
 - Key Code
 1. Tools Layer Entry (`largetmodel/utis/tools_manager.py`)
 2. Model interface layer (`largetmodel/utis/large_model_interface.py`)
 - Code Analysis
- 3.1 Configuring the Offline Large Model
 - 3.1.1 Configuring the LLM Platform (`yahboom.yaml`)
 - 3.1.2 Configuring the Model Interface (`larget_model_interface.yaml`)
- 3.2 Starting and Testing the Function
4. Common Problems and Solutions
 - 4.1 Very Slow Responses

1. Concept Introduction

1.1 What is "Visual Understanding"?

In the `largetmodel` project, the **multimodal visual understanding** feature enables robots to go beyond simply "seeing" a matrix of pixels and truly "understand" the content, objects, scenes, and relationships within an image. This is like giving robots a pair of thinking eyes.

The core tool for this feature is `**seewhat``. When a user issues a command like "see what's here," the system invokes this tool, triggering a series of background operations that ultimately provide the user with AI-generated analysis of the live image in natural language.

1.2 Implementation Principle Overview

The basic principle is to input two different types of information—**image (visual information)** and **text (linguistic information)**—into a powerful multimodal large model (such as LLaVA).

1. **Image Encoding:** The model first uses a vision encoder to convert the input image into computer-interpretable digital vectors. These vectors capture image features such as color, shape, and texture.
2. **Text Encoding:** Simultaneously, the user's question (e.g., "What's on the table?") is converted into a text vector.
3. **Cross-modal Fusion:** The most critical step is the model fusing the image vector and the text vector in a special "attention layer." Here, the model learns to "focus" on the parts of the image relevant to the question. For example, when asked about "table," the model will pay more attention to areas of the image that embody the characteristics of a table.
4. **Answer Generation:** Finally, a large language model (LLM) generates a descriptive text answer based on this fused information.

Simply put, this involves **highlighting the corresponding parts of the image with text, and then describing the highlighted parts with language.**

2. Code Explanation

Key Code

1. Tools Layer Entry (largemodel/utis/tools_manager.py)

The `seewhat` function in this file defines the execution flow of the tool.

```
# From largemodel/utis/tools_manager.py

class ToolsManager:
    # ...

    def seewhat(self):
        """
        Capture camera frame and analyze environment with AI model.
        捕获摄像头画面并使用AI模型分析环境。

        :return: Dictionary with scene description and image path, or None if
        failed.
        """
        self.node.get_logger().info("Executing seewhat() tool")
        image_path = self.capture_frame()
        if image_path:
            # Use isolated context for image analysis. / 使用隔离的上下文进行图像分
            析。

            analysis_text = self._get_actual_scene_description(image_path)

            # Return structured data for the tool chain. / 为工具链返回结构化数据。
            return {
                "description": analysis_text,
                "image_path": image_path
            }
        else:
            # ... (Error handling)
            return None

    def _get_actual_scene_description(self, image_path, message_context=None):
        """
        Get AI-generated scene description for captured image.
        获取捕获图像的AI生成场景描述。

        :param image_path: Path to captured image file.
        :return: Plain text description of scene.
        """
        try:
            # ... (构建Prompt)

            # Force use of a plain text system prompt with a clean, one-time
            context. / 强制使用纯文本系统提示和干净的一次性上下文。
            simple_context = [{
                "role": "system",
                "content": "You are an image description assistant. ..."
            }]

            result = self.node.model_client.infer_with_image(image_path,
            scene_prompt, message=simple_context)
```

```

        # ... (处理结果)
        return description
    except Exception as e:
        # ...

```

2. Model interface layer

(`largemodel/utils/large_model_interface.py`)

The `infer_with_image` function in this file is the unified entry point for all image understanding tasks. It is responsible for calling the specific model implementation according to the configuration.

```

# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_image(self, image_path, text=None, message=None):
        """Unified image inference interface. / 统一的图像推理接口."""
        # ... (准备消息)
        try:
            # 根据 self.llm_platform 的值, 决定调用哪个具体实现
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages,
image_path=image_path)
            elif self.llm_platform == 'tongyi':
                # ... 调用通义模型的逻辑
                pass
            # ... (其他平台的逻辑)
        # ...
        return {'response': response_content, 'messages': self.messages.copy()}

```

Code Analysis

This feature's implementation involves two main layers: the tool layer defines the business logic, and the model interface layer is responsible for communicating with the large language model. This layered design is key to achieving platform versatility.

1. Tool Layer (`tools_manager.py`):

- The `seewhat` function is the core business of the visual understanding function. It encapsulates the entire "seeing" action process: first, it calls the `capture_frame` method to obtain an image, then calls `_get_actual_scene_description` to prepare a prompt for the model to analyze the image.
- The most critical step is calling the `infer_with_image` method of the model interface layer. It does not care about the underlying model; it only passes the two core data elements, "image" and "analysis instructions," to the model interface layer.
- Finally, it packages the analysis results (plain text descriptions) received from the model interface layer into a structured dictionary and returns them. This allows upper-layer applications to easily use the analysis results.

2. Model Interface Layer (`large_model_interface.py`):

- The `infer_with_image` function acts as a "dispatching center." Its primary responsibility is to check the current platform configuration (`self.llm_platform`) and, based on the

configuration, dispatch tasks to specific handlers (such as `ollama_infer` or `tongyi_infer`).

- This layer is key to adapting to different AI platforms. All platform-specific operations (such as data encoding and API call formats) are encapsulated within their respective handlers.
- In this way, the business logic code in `tools_manager.py` remains unchanged to support a variety of different large-model backend services. It simply interacts with the unified, stable `infer_with_image` interface.

In summary, the `seewhat` tool's execution flow demonstrates a clear separation of responsibilities: `ToolsManager` defines the "what" (acquiring an image and requesting analysis), while `model_interface` defines the "how" (selecting the appropriate model platform based on the current configuration and interacting with it). This makes the tutorial's analysis universal, ensuring the core code logic remains consistent regardless of whether the user is online or offline.

3.1 Configuring the Offline Large Model

3.1.1 Configuring the LLM Platform (`yahboom.yaml`)

This file determines which large model platform the `model_service` node loads as its primary language model.

1. Open the file in the terminal:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

2. Modify/Confirm `llm_platform`:

```
model_service:                                #模型服务器节点参数 Model server node
parameters
  ros__parameters:
    language: 'en'                             #大模型接口语言 Large Model Interface
    Language
    useolinetts: True                          #文字模式下此项无效，可忽略 This item is
invalid in text mode and can be ignored

    # 大模型配置 Large model configuration
    llm_platform: 'ollama'                     # 关键：确保这里是 'ollama' Key: Make
sure it's 'ollama'
    regional_setting : "international"
```

3.1.2 Configuring the Model Interface (`large_model_interface.yaml`)

This file defines which vision model to use when the `ollama` platform is selected.

1. Open the file in Terminal

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

2. Find the ollama related configuration

```
#.....  
## 离线大模型 (Offline Large Language Models)  
# Ollama Configuration  
ollama_host: "http://localhost:11434" # Ollama server address  
ollama_model: "llava" # Key: Change this to the multimodal model you  
downloaded, such as "llava"  
#.....
```

Note: Please make sure that the model specified in the configuration parameters (such as `llava`) can handle multimodal input.

3.2 Starting and Testing the Function

Note: Due to performance limitations, this example cannot be run on the Jetson Orin Nano 4GB. To experience this function, please refer to the corresponding section in <Online Large Model (Voice Interaction)>

1. **Start the `largemodel` main program:**

Open a terminal and run the following command:

```
ros2 launch largemodel largemodel_control.launch.py
```

2. After successful initialization, say the wake-up word and begin asking questions like: "What do you see?" or "Describe your current environment."

3. **Observe the results:**

In the first terminal where the main program is running, you will see log output showing that the system receives the text command, invokes the `seewhat` tool, and ultimately prints the text description generated by the LLaVA model. The speaker will also announce the generated results.

4. Common Problems and Solutions

4.1 Very Slow Responses

Problem: After asking a question, it takes a long time for the voice response to arrive.

Solution:

The inference cost of a multimodal model is much higher than that of a text-only model, so higher latency is normal.

1. **Use a Smaller Model:** In `large_model_interface.yaml`, try using the lighter `llava` model version.